

AI Fact-Checking System

Group ID
COMP90042 Project 2026

Abstract

We present a resource-aware fact-checking system for climate-related claims. The goal is to obtain a strong performance–time trade-off under limited compute, using a standard Colab-style runtime as a concrete example of this constraint. Our system uses staged evidence processing: cheap sparse fusion keeps broad recall, embedding and factual-cue reranking compresses the evidence pool, a bounded cross-encoder pass selects submitted evidence, and a lightweight structured-evidence classifier predicts the final verdict. Among our Colab-feasible variants, this design gives the strongest constrained result, obtaining 0.2021 evidence F-score, 0.5000 claim accuracy, 0.2879 harmonic mean, and 0.4659 classifier macro-F1. The main finding is that evidence rank and shallow factual cues should be preserved through the pipeline rather than discarded by plain evidence concatenation.

1 Introduction

Climate misinformation is a high-impact setting for automated fact checking: claims are often short, rhetorically compressed, and require comparison against specific scientific evidence. A useful system must therefore do more than predict a label; it must also retrieve evidence that explains the prediction. In practical settings, fact-checking systems often need to operate under limited compute resources rather than assuming access to large offline indexes or expensive all-pair neural scoring. We instantiate this constraint with a free-tier Google Colab setting: the system is expected to run end-to-end without external services, saved checkpoints, or precomputed ranked artifacts. This resource boundary changes the modelling problem. A method that is accurate but scores all claim–evidence pairs with a large neural model is not a practical solution under this setting. The central research question is therefore how to balance ev-

idence recall, claim-label accuracy, and runtime within a limited compute budget.

Draft note: I have collected, organized, and grouped a set of relevant papers for this section. Please continue the related-work writing based on this structure and build on the material below.

Existing work can be organized into four areas:

1. **Evidence-Based Fact Checking** FEVER-style work defines fact checking as a joint evidence retrieval and verdict prediction task (Thorne et al., 2018). Climate-specific datasets such as CLIMATE-FEVER show that real climate claims introduce subtle domain and evidence-retrieval challenges (Diggelmann et al., 2020). This line of work mainly defines the task and evaluation setting.
2. **Large-Scale Evidence Retrieval** Sparse retrieval methods such as BM25 are fast and scalable (Robertson et al., 1994), and reciprocal rank fusion provides a simple unsupervised way to combine several ranked lists (Cormack et al., 2009). These methods are useful for high-recall candidate generation, but they mostly rely on lexical matching.
3. **Neural Semantic Reranking** Dense retrievers and sentence embeddings improve matching when lexical overlap is weak (Karpukhin et al., 2020; Reimers and Gurevych, 2019), while cross-encoder rerankers can model claim–evidence interactions more directly (Nogueira and Cho, 2019). Their limitation is cost: they are more suitable after a sparse candidate filter than as full-corpus scorers.
4. **Evidence Aggregation and Factual Cue Fusion** Evidence aggregation work studies how retrieved passages should be combined before label prediction (Zhou et al., 2019). Hybrid fact-checking systems further show that lexical, semantic, and shallow factual signals can

be complementary rather than mutually exclusive (Lee et al., 2018; Padia et al., 2018).

Our design follows this literature but is driven by the Colab constraint. The first stage is a broad sparse candidate module. It fuses word matching, character matching, structured factual cues, and pseudo-relevance feedback with fixed reciprocal rank fusion weights. This module is unsupervised, cheap, and broad enough to preserve evidence coverage before neural scoring. The second and third stages are hierarchical rerankers: the broad pool is compressed into a smaller classifier context, and a bounded cross-encoder pass then selects the submitted evidence. This separation lets the system use expensive neural comparisons only where they are most useful. The reranking implementation combines MiniLM embedding inner products with hand-built factual features. The idea is multi-scale feature fusion: embeddings capture paraphrase-level semantic similarity, while shallow cues capture exact words, numbers, years, negation, source rank, and length shape. The classifier then uses a structured evidence representation, which preserves evidence rank and factual cue tokens before applying a simple TF-IDF logistic regression model. This keeps classification lightweight while passing forward the signals that earlier stages found useful.

Our contributions are fourfold. First, we design a staged fact-checking pipeline that balances runtime and model quality under a strict Colab resource budget, achieving the strongest result among our Colab-feasible tutorial variants. Second, we propose a multi-gate unsupervised sparse retrieval scheme for broad candidate generation. Third, we design a multi-scale reranking method that fuses shallow factual cues with embedding similarity and show that these cues improve retrieval-context quality. Fourth, we provide a systematic experimental analysis of sparse gates, fusion routes, top-64 selection, top-3 evidence selection, and classifier sensitivity.

2 Method

2.1 Pipeline Overview

Let $\mathcal{C}$ be the set of claims and $\mathcal{E}$ the evidence corpus. The task requires both a verdict label and a small evidence set for each claim. A direct neural approach would score every pair $(c, e) \in \mathcal{C} \times \mathcal{E}$, which is not practical for a large corpus under lim-

ited compute. We therefore use a staged pipeline:

$$\mathcal{E} \rightarrow \mathcal{A}(c) \rightarrow \mathcal{B}(c) \rightarrow \mathcal{D}(c),$$

where $\mathcal{A}(c)$ is a broad sparse candidate pool, $\mathcal{B}(c)$ is a smaller reranked context pool, and $\mathcal{D}(c)$ is the submitted evidence set. The design principle is that early stages should be cheap and high-recall, while later stages may use stronger semantic models because they operate on much smaller sets.

The pipeline is organized into four components. First, a multi-gate sparse retriever combines several lexical and factual views of the same corpus. Second, a multi-scale feature fusion reranker combines dense semantic similarity with shallow factual alignment features. Third, a bounded neural selector uses a cross-encoder only on a prefiltered subset. Finally, a structured-evidence classifier converts the selected evidence context into a verdict label.

2.2 Multi-Gate Sparse Candidate Retrieval

The sparse stage is designed for recall. The BM25 equations below define the shared scoring operator used inside each sparse gate; they are not meant to say that the system has only one retrieval view. The submitted candidate module uses four elementary gates—word, character, structured factual cues, and pseudo-relevance feedback—and the diagnostics also report the final RRF-fused candidate view as a fifth sparse view. For term t in document d , the evidence-side BM25 score is

$$\text{idf}_t = \log \left(\frac{N - df_t + 0.5}{df_t + 0.5} + 1 \right),$$

$$\text{BM25}_{d,t} = \text{idf}_t \frac{(k_1 + 1)f_{d,t}}{f_{d,t} + k_1(1 - b + b|d|/|d|)}.$$

The query side remains a count vector, so each gate score is a sparse dot product,

$$s_g(c, d) = q_g(c)^\top \text{BM25}_g(d).$$

In the submitted notebook this is implemented with SciPy/NumPy sparse matrix operations rather than a Python postings loop. The expensive document-frequency counts, BM25 reweighting, and query-document dot products are therefore handled by optimized C/C++ numerical kernels and vectorized array operations. This implementation choice substantially accelerates the broad candidate stage while keeping the mathematical retrieval signal equivalent to the sparse gates described above.

The elementary gates are complementary rather than redundant. The word gate captures lexical content and short phrase matches. The character gate is robust to word form variation and partial string overlap. The structured gate isolates factual surface cues such as numbers, years, capitalized terms, and negation markers. The pseudo-relevance feedback gate expands the claim using terms from the initial lexical retrieval results. The fifth sparse view reported in the figures is the fused candidate pool, obtained by combining these elementary ranked lists with weighted reciprocal rank fusion:

$$S_{\text{RRF}}(c, d) = \sum_{g \in G} \frac{\lambda_g}{K_{\text{RRF}} + r_g(c, d)},$$

where $r_g(c, d)$ is the rank of evidence d under gate g , and missing documents contribute zero. The fusion weights λ_g are fixed by the notebook configuration and are treated as a reliability prior: stronger standalone gates should receive larger weights, while weaker but complementary signals can remain with small non-zero weights. In our diagnostics the structured gate has low standalone recall, but a controlled ablation shows that adding it back to the sparse fusion improves the fused top-500 recall. This rank-level fusion avoids direct score calibration across heterogeneous sparse views.

Figures 1 and 2 support the candidate-stage choice. The sparse fusion improves the top-500 operating point over any single sparse source, while staged pruning avoids the infeasible cost of scoring the full evidence corpus with later neural components.

2.3 Multi-Scale Feature Fusion

The reranking stage is the main feature-fusion component of the system. It combines signals at different scales: dense semantic similarity, sparse source rank, word overlap, numeric agreement, negation compatibility, and length shape. The dense semantic score is computed with a MiniLM sentence encoder. After mean pooling and ℓ_2 normalization, cosine similarity reduces to an inner product:

$$s_{\text{emb}}(c, d) = h(c)^\top h(d).$$

The shallow factual features are summarized as

$$\phi(c, d) = [\phi_{\text{rank}}, \phi_{\text{lex}}, \phi_{\text{num}}, \phi_{\text{neg}}, \phi_{\text{len}}, s_{\text{emb}}].$$

Here ϕ_{rank} represents sparse source rank, ϕ_{lex} represents claim–evidence word overlap, ϕ_{num} represents number matching, ϕ_{neg} represents negation

compatibility, and ϕ_{len} represents length mismatch. A lightweight gradient-boosted matching model estimates $p_{\text{feat}}(c, d)$, the probability that a candidate is gold evidence. The final reranking score is

$$S_{\text{ctx}}(c, d) = \alpha \tilde{s}_{\text{emb}}(c, d) + (1 - \alpha) p_{\text{feat}}(c, d),$$

where $\tilde{s}_{\text{emb}}$ is normalized within the claim-specific candidate pool. This formulation makes the intended complementarity explicit: embedding similarity handles semantic relatedness, while factual features guard against mismatched numbers, negation direction, and weak lexical grounding.

Figures 3 and 4 justify the multi-scale feature fusion design. The model is not relying on hand cues as standalone rules; the diagnostics show that factual cues complement semantic similarity and improve the selected classifier context.

2.4 Bounded Neural Evidence Selection

The submitted evidence list uses a stricter operating point than the classifier context. A cross-encoder is more expressive than independent embeddings because it jointly encodes the claim and evidence, but it is also much more expensive. The notebook therefore applies the cross-encoder only to a pre-filtered subset selected by sparse and embedding ranks. Its raw score is normalized within each claim’s scored subset.

The final submitted-evidence score is a fusion of cross-encoder confidence, embedding confidence, embedding rank, and sparse source rank:

$$S_{\text{sub}}(c, d) = \beta_1 \tilde{s}_{\text{CE}} + \beta_2 \tilde{s}_{\text{emb}} + \frac{\beta_3}{\tau + r_{\text{emb}}} + \frac{\beta_4}{\tau + r_{\text{src}}}.$$

The coefficients are fixed by the notebook configuration. Candidates not scored by the cross-encoder receive no cross-encoder contribution, but they can still be selected through embedding and source-rank terms. This keeps the neural step bounded without discarding useful candidates from the broader sparse pool.

2.5 Structured Evidence Classification

The classifier predicts the final verdict from a bounded ranked evidence context produced by the reranker. Instead of treating all retrieved passages as an unordered block of text, the notebook constructs a structured text input:

$$x_c = [\text{claim}; z_1, z_1, \dots, z_m, z_m, z_{m+1}, \dots, z_L],$$

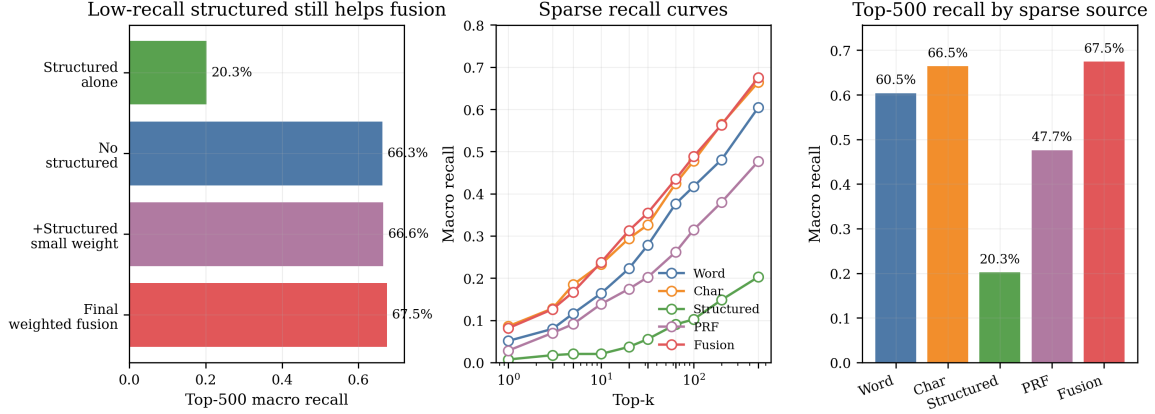


Figure 1: Sparse fusion gives the best top-500 recall, and the structured gate is retained for complementarity. Character matching is the strongest single sparse source and receives the largest weight. Although structured cues are weak alone, the ablation panel shows that removing them lowers the fused top-500 recall under a controlled sparse-fusion setting, so the final system keeps structured evidence with a small non-zero weight.

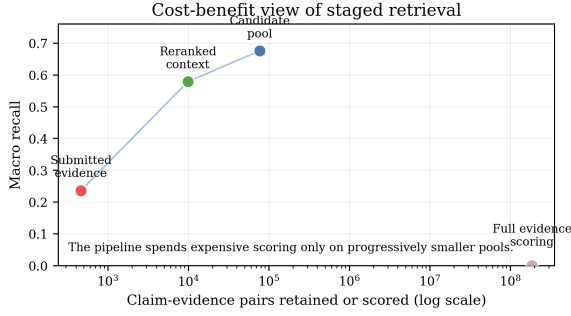


Figure 2: Staged pruning keeps recall while reducing pair count. The candidate and reranked-context pools retain most recoverable evidence while avoiding the orders-of-magnitude cost of full-corpus neural scoring.

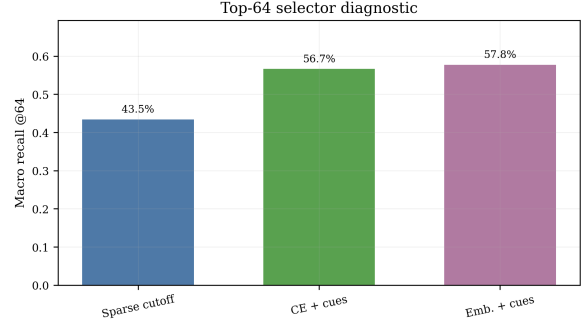


Figure 3: Embedding plus factual cues is the best top-64 selector. It reaches 57.8% macro recall, above sparse cutoff and cross-encoder-plus-cue variants, justifying its use for the classifier evidence context.

where the highest-ranked evidence lines are repeated to increase their TF-IDF weight. Each evidence line also receives a rank prefix and compact cue tokens. The cues encode word-overlap bucket, numeric match or mismatch, and negation match or mismatch. They are not rules; they are ordinary tokens that allow the classifier to learn whether these factual signals are predictive.

The verdict model remains deliberately lightweight. Let v_c be the TF-IDF vector of the structured context for claim c . A one-vs-rest logistic model learns one binary classifier per label:

$$P(y = k \mid c) = \sigma(\theta_k^\top v_c + b_k).$$

The final label is selected by the largest one-vs-rest decision score. This keeps the classifier computationally small while preserving the rank and factual signals produced by the retrieval stages.

Stage	F-score	Macro recall	Hit-any
Sparse top-500	0.0083	0.6754	0.9026
Context top-64	0.0517	0.5784	0.8636
Submitted top-3	0.2021	0.2355	0.4675

Table 1: Retrieval metrics from the self-generated notebook artifacts.

3 Results

The final notebook was run in Colab with GPU enabled. The course evaluation reports three official development metrics: evidence retrieval F-score F , claim classification accuracy A , and the H-score $H = 2FA/(F + A)$, the harmonic mean used as the main leaderboard metric. Table 1 reports the main retrieval operating points and Table 2 reports the reproduced official and diagnostic development results.

Figure 5 explains the final classifier result be-

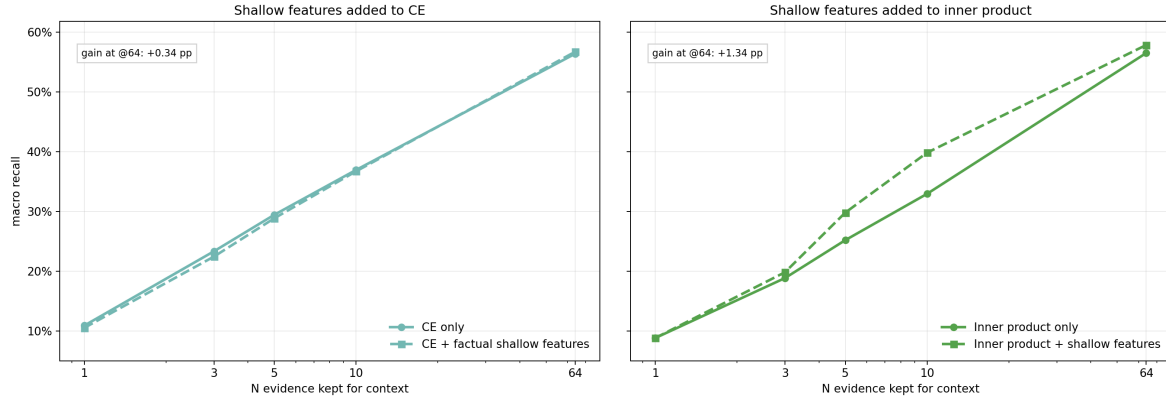


Figure 4: Factual cues preserve or improve semantic ranking. The gain is small for the already strong cross-encoder but clearer for embedding inner product, supporting feature fusion rather than a purely semantic context selector.

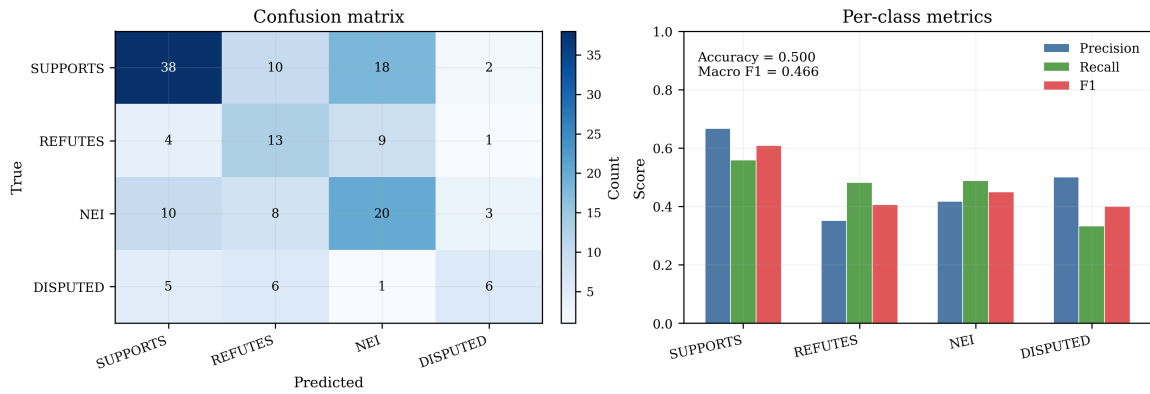


Figure 5: Classifier errors concentrate among the three main labels. The final Colab run performs best on SUPPORTS, while DISPUTED remains difficult because it is rare and often depends on conflicting evidence.

Metric	Value
Evidence Retrieval F-score (F)	0.2021
Claim Classification Accuracy (A)	0.5000
Harmonic Mean of F and A (H)	0.2879
Top-3 macro recall (diagnostic)	0.2355
Classifier macro-F1 (diagnostic)	0.4659

$$H = 0.2879.$$

Table 2: Final Colab-reproduced development result. F , A , and $H = 2FA/(F + A)$ are the official course metrics; the other rows are diagnostics.

yond the aggregate accuracy. The dominant remaining errors are confusions among SUPPORTS, REFUTES, and NEI; the DISPUTED class is evaluated on only 18 development examples.

Table 3 justifies the classifier input design. Plain context reached 0.4351 accuracy and 0.3762 macro-F1, while enhanced context reached 0.4935 accuracy and 0.4544 macro-F1 at $C = 0.25$. A diagnostic run at $C = 0.125$ reached 0.5195 accuracy and 0.4841 macro-F1. The primary reported result, however, is the Colab-reproduced official run in Table 2, where $F = 0.2021$, $A = 0.5000$, and

Context format	C	Acc.	Macro-F1
Plain	0.25	0.4351	0.3762
Rank-aware	0.25	0.4351	0.3749
Top-weighted	0.25	0.4416	0.3938
Rank-weighted	0.25	0.4351	0.3882
Enhanced	0.25	0.4935	0.4544
Enhanced	0.125	0.5195	0.4841

C	Acc.	Macro-F1
0.125	0.5195	0.4841
0.25	0.4935	0.4544
0.5	0.4740	0.4263
1.0	0.4675	0.4184
2.0	0.4416	0.3825
4.0	0.4545	0.3913
8.0	0.4481	0.3925

Table 3: Classifier ablations. The left subtable shows that rank-aware, top-weighted, factual-cue-enhanced context improves over plain concatenation; the right subtable shows that the enhanced classifier is strongest at small regularization strength $C = 0.125$ in the diagnostic sweep.

References

- Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. 2009. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759.
- Thomas Diggelmann, Jordan Boyd-Graber, Jannis Buellian, Massimiliano Ciaramita, and Markus Leippold. 2020. CLIMATE-FEVER: A dataset for verification of real-world climate claims. *arXiv preprint arXiv:2012.00614*.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781.
- Nayeon Lee, Chien-Sheng Wu, and Pascale Fung. 2018. Improving large-scale fact-checking using decomposable attention models and lexical tagging. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 1133–1138.
- Rodrigo Nogueira and Kyunghyun Cho. 2019. Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*.
- Ankur Padia, Francis Ferraro, and Tim Finin. 2018. Team UMBC-FEVER: Claim verification using semantic lexical resources. In *Proceedings of the First Workshop on Fact Extraction and VERification*, pages 161–165.
- Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 3982–3992.
- Stephen E. Robertson, Steve Walker, Susan Jones, Micheline Hancock-Beaulieu, and Mike Gatford. 1994. Okapi at TREC-3. In *Proceedings of the Third Text REtrieval Conference*, pages 109–126.
- James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. 2018. FEVER: A large-scale dataset for fact extraction and verification. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 809–819.
- Jie Zhou, Xu Han, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. 2019. GEAR: Graph-based evidence aggregating and reasoning for fact verification. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 892–901.